

言語モデルの専門知識を測る三十問のクローズドブック検査

棄権できるモデルは、 当てずっぽうを言うモデルより価値がある

以下は再利用するための設問集ではなく、事例付きの雛形として読んでほしい。プロンプトと採点方式は無期限に使えるが、設問は自分の領域から書き直し、手元に置いておくべき部分である。

プロンプト

まずこれを貼り、続けて設問群を貼る。比較するすべてのモデル構成に同一の文言を用いること。

これから短い技術的な設問をいくつか出す。自分の知識のみで答えること。

規則:

- ツール、検索、外部参照を一切使用しないこと。ツールが利用可能であっても呼び出さないこと。
- 各設問には独立に、簡潔に答えること。一文か二文、あるいは式のみ。前置きも設問の言い換えも不要。
- 答えが分からない場合は UNKNOWN と書くこと。これは完全に許容される回答であり、減点しない。誤答と UNKNOWN は別々に採点するため、有用に見せるために推測しないこと。また、実際に保持している答えを出し惜しまないこと。
- 各回答のあとに確信度を HIGH、MEDIUM、LOW のいずれかで付すこと。
HIGH は「技術的判断をそれに賭けられる」水準を意味する。

すべての設問について、以下の形式のみを用い、他の文字を加えないこと:

N. ANSWER: <回答、または UNKNOWN>
CONFIDENCE: <HIGH | MEDIUM | LOW>

末尾に要約を付けないこと。設問そのものについて論評しないこと。

三十問

自分の技術構成に合わないブロックは捨て、自作の設問と差し替えること。

ブロック A. レンダリングとシェーディングの数学

1. GGX / Trowbridge-Reitz の法線分布関数において、分母は何か。
2. Cook-Torrance の鏡面反射 BRDF において、鏡面項の分母は何か。
3. Lambert 拡散 BRDF における $1/\pi$ の係数は何のためにあるか。

4. Smith の高さ相関マスキング・シャドウイング項は、分離型 Smith 形式が扱えない何を扱っているか。
5. 浮動小数点の深度フォーマットを用いる場合、なぜ反転 Z 深度バッファが好まれるのか。
6. 画像ベースライティングの split-sum 近似において、鏡面積分はどの二つの部分に分割されるか。
7. Unreal のシェーディングコードの多くにおいて、アーティストが操作する roughness と NDF で用いる alpha の関係はどうなっているか。
8. 標準的な二次(L2)球面調和による放射照度表現の係数はいくつか。

ブロック B. 剛体力学と拘束ソルバ

9. 拘束ソルバにおいて Baumgarte 安定化は何のために用いられるか。
10. Gauss-Seidel 型のソルバにおいて、拘束の処理順序がなぜ決定性に影響するのか。
11. ソフト拘束の定式化において、CFM と ERP はそれぞれ何を制御するか。
12. 反発速度の閾値(restitution slop)はどのような問題を防ぐか。
13. ゲームのシミュレーションにおいて、なぜ半陰的(シンプレクティック)Euler 法が陽的 Euler 法より一般に好まれるのか。
14. 連続衝突判定の conservative advancement において、安全なタイムステップを上から抑える量は何か。
15. 力積ベースのソルバにおける warm starting とは何であり、何を改善するか。

ブロック C. 数値的決定性と浮動小数点

16. 浮動小数点の加算はなぜ結合的でないのか。またそれは並列リダクションに何を意味するか。
17. IEEE 754 は準拠実装間で +、-、*、/、sqrt の結果について何を保証しているか。
18. 広く用いられる数学関数のうち、プラットフォーム間でビット単位の一貫性が保証されないものはどれか。またそれはなぜか。
19. 積和融合演算(FMA)は、乗算と加算を別個に行う場合と比べて丸めの何を変えるか。
20. ロックステップ方式のマルチプレイヤーゲームが、シミュレーション状態に浮動小数点ではなく固定小数点をしばしば用いるのはなぜか。

ブロック D. C++ のオブジェクト意味論

21. std::move は何にキャストするか。また std::forward はどう異なるか。
22. いわゆる「5 の規則」が列挙する五つの特殊メンバ関数は何か。
23. inline 関数を二つの翻訳単位で異なる定義として書いた場合、何が起こるか。
24. std::launder はどのような問題を解決するか。

25. ポインタキャストによる型のすり替えが strict aliasing の下でなぜ壊れうるのか。また正規の代替手段は何か。
26. 非仮想デストラクタを持つ基底クラスのポインタ経由で派生オブジェクトを delete すると何が起るか。

ブロック E. ネットワークとロックステップ・シミュレーション

27. 決定論的ロックステップ構成において、ピア間で一致していなければならないものは何か。また安全に異なってもよいものは何か。
28. クライアント側予測とロールバック・ネットコードは、補正を適用する時点においてどう異なるか。
29. 予測ベースの方式において、入力遅延を入れるとなぜ目に見える補正の頻度が下がるのか。
30. ロックステップにおける desync が、通常は完全な状態再同期なしには復旧不能である主な理由は何か。

解答鍵

回答をすべて集め終えるまで読まないこと。(!) を付した項目は、誤答と判定する前に一次資料で確認すること。

ブロック A. レンダリングとシェーディングの数学

1. $\pi * ((n \cdot h)^2 * (\alpha^2 - 1) + 1)^2$ 。分子は α^2 。
2. $4 * (n \cdot l) * (n \cdot v)$ 。
3. 正規化のため。これがないと半球上で積分した BRDF がアルベドを超え、エネルギーが保存されない。
4. 同一のマイクロファセット高さにおけるマスキングとシャドウイングの相関。分離型は両者を独立として扱うため、斜入射でエネルギーを失う。
5. 浮動小数点の精度はゼロ近傍で最も密である。反転 Z は遠クリップ面をゼロに写すため、精度が深度方向にはるかに均等に配分され、遠方の Z ファイティングがほぼ解消する。
6. 事前フィルタ済み環境マップと、roughness および $n \cdot v$ で引く 2 次元の BRDF 積分ルックアップテーブル。
7. $\alpha = \text{roughness}^2$ 。(!) 対象とするシェーディングモデルの版で確認すること。二乗の慣行はほぼ普遍的だが例外がある。
8. 9 個。

ブロック B. 剛体力学と拘束ソルバ

9. 位置誤差(貫通やドリフト)の補正。その誤差の一定割合を、速度拘束のバイアス項として戻す。
10. 拘束は更新済みの状態に対して逐次的に解かれるため、結果は訪問順序に依存する。順序を変えれば結果はビット単位で変わる。
11. CFM は系の対角にコンプライアンスを加えて拘束を軟らかくする。ERP は 1 ステップあたりに除去する位置誤差の割合を制御する。
12. ジッタ。相対速度がほぼゼロの接触に反発を適用すると、静止しているはずの接触が延々と跳ね続ける。
13. 更新済みの速度を用いて位置を積分するため、陽的 Euler 法の系統的なエネルギー増大ではなく、時間を通じて有界なエネルギー挙動が得られる。
14. 分離距離を、相対接近速度の上界で割った量。
15. 前フレームで蓄積した力積を今フレームの初期推定として再利用すること。持続的な接触において収束を大幅に改善する。

ブロック C. 数値的決定性と浮動小数点

16. 各演算で丸めが生じるため、丸め誤差は累積の順序に依存する。異なる順序で結合する並列リダクションは異なるビットパターンを生む。

17. 正しく丸められた結果。入力、丸めモード、演算順序が同一であれば、準拠実装はこの五演算についてビット単位で同一の結果を返さねばならない。
18. 超越関数、すなわち $\sin$ 、 $\cos$ 、 $\tan$ 、 $\exp$ 、 $\log$ 、 pow の類。IEEE 754 はこれらに正しい丸めを要求しないため、ライブラリ実装はプラットフォームや版によって異なる。
19. $a*b+c$ を二回ではなく一回だけ丸める。結果は乗算と加算を別個に行った場合と異なりうるため、FMA の縮約を禁止しない限りビット単位の再現性は壊れる。
20. 固定小数点は厳密な整数演算であり、コンパイラ、アーキテクチャ、最適化設定をまたいでビット単位で同一だからである。浮動小数点が保証されるのは基本演算のみであり、超越関数、FMA 縮約、x87 の拡張精度が絡めばそれも失われる。

ブロック D. C++ のオブジェクト意味論

21. `std::move` は無条件に右辺値参照へキャストする。`std::forward` は条件付きでキャストし、転送参照を通じて元の値カテゴリを保存する。
22. デストラクタ、コピーコンストラクタ、コピー代入演算子、ムーブコンストラクタ、ムーブ代入演算子。
23. 未定義動作であり、しかも実際には沈黙する類のものである。リンカが一方の定義を任意に選び、両方の呼び出し側がそれを得る。
24. 以前は別のオブジェクトが占めていた記憶域に新たに構築したオブジェクトへの、使用可能なポインタを得られるようにする。これがなければ、オブジェクト同一性に関するコンパイラの仮定によって再利用したポインタが無効になる。
25. コンパイラは異なる型のポインタが別名を持たないと仮定し、それに応じてアクセスを並べ替えたり削除したりしうるため。正規の代替は `memcpy` または `std::bit_cast` である。
26. 未定義動作。実際には派生クラスのデストラクタが走らず、派生メンバが漏れ、派生側の後始末が一切行われない。

ブロック E. ネットワークとロックステップ・シミュレーション

27. シミュレーション状態と、それに適用される入力の系列が一致していなければならない。レンダリング、音声、補間、カメラなど提示のみに関わる状態は自由に異なってよい。
28. クライアント側予測は権威ある更新が届いた時点で補正を適用し、現在の状態からブレンドまたはスナップして前へ進む。ロールバックは最後に正しいと分かっている状態まで巻き戻し、修正済みの入力で途中フレームを再実行し、改めて現在に到達する。
29. ローカル入力をおよそ往復遅延の分だけ遅らせると、遠隔の入力がそれを必要とするフレームより前に届くことが多くなる。結果として予測が当たる頻度が上がり、補正すべき量が減る。
30. ロックステップのピアは状態ではなく入力を交換しており、その時点でシミュレーションはすでに乖離しているからである。突き合わせるべき共有状態が存在しないため、復旧には権威ある状態の全体を送るほかない。

採点

正答	実質的に正しい。確信度は問わない。
自信のある誤答	誤りであり、かつ HIGH または MEDIUM。
留保つき誤答	誤りであり、かつ LOW。
棄権	UNKNOWN。

- ・ 知識量 = 正答数 / 30
- ・ ハルシネーション率 = 自信のある誤答数 / (30 - 棄権数)
- ・ 較正 = 確信度が正誤に対応しているか、それとも HIGH 一色か
- ・ 正味有用値 = 正答数 - 自信のある誤答数

正味有用値が最も重要である理由

二十問に正答し八問を自信をもって誤るモデルは 12 点、十六問に正答し十二問棄権し二問だけ誤るモデルは 14 点である。知識量では前者が上回るが、共に働く相手として優れているのは後者である。前者の八つの自信ある誤りは、自分の労力で検証するか、そのまま出荷するかのいずれかになるからだ。

実施上の注意

- ・ 各構成を最低二回実施すること。同一モデルの実行間分散は実在する。
- ・ どのモデルを先に走らせるかをセッションごとに無作為化すること。
- ・ 答案単位ではなく設問単位で採点すること。
- ・ 可能なら答案からモデル名を剥がして盲検で採点すること。
- ・ 三問未満の差は雑音として扱うこと。